

Plano — Exportação completa dos dados do evento pelo indicador REC

Data: 2026-09-03

Status: planejado

Tipo: funcionalidade

Escopo: SmartEvents Desktop, persistência dos metadados da EP e testes do pacote exportado

Objetivo

Permitir que o operador exporte os dados persistidos de um evento para um único pacote ZIP, diretamente pelo indicador de gravação **REC**. O pacote deve ser compreensível por uma pessoa, utilizável por Excel e ferramentas de análise, auditável por máquina e seguro para ser entregue ao usuário final.

A exportação deve incluir KPIs, medições de VIPs, alarmes, alertas, configuração do evento e as informações normalizadas provenientes da EP. Para os KPIs, o operador poderá escolher entre um arquivo consolidado ou arquivos por dia e poderá manter todas as tecnologias juntas ou separá-las por família/tipo de coleta.

Decisão sobre fases

A implementação será tratada como **uma única fase**.

A funcionalidade forma uma entrega vertical única: o contrato do pacote determina as consultas e os testes; o motor de exportação precisa estar ligado a um job em background; e o job só entrega valor quando pode ser iniciado e acompanhado pelo modal do **REC**. Separar essas partes em fases independentes criaria estados intermediários sem utilidade para o operador, como uma API sem interface ou um botão sem geração confiável do arquivo.

A fase única terá passos internos ordenados para reduzir risco:

1. estabilizar o contrato do pacote e o modelo da EP;
2. implementar snapshot, consultas e geração dos arquivos;
3. expor o job pela API Python/JavaScript;
4. substituir o fluxo direto de limpeza do **REC** pelo modal de dados;
5. validar integridade, desempenho, interface e empacotamento do executável.

Decisões que valem para toda a implementação

1. Horário de Brasília é a única referência de calendário

Toda conversão de data local, agrupamento por dia e exibição de período deve usar

```
ZoneInfo("America/Sao_Paulo")
```

.

Não usar o timezone configurado no Windows e não usar um deslocamento fixo como `UTC-03:00`.

`America/Sao_Paulo` representa o horário de Brasília e continua correto para datas históricas que possam ter regras diferentes de horário de verão.

O pacote preservará simultaneamente:

- `timestamp_utc` : instante normalizado em UTC, com sufixo `Z` ;
- `timestamp_brasilia` : mesmo instante no horário de Brasília, com offset explícito;
- `data_brasilia` : `YYYY-MM-DD` , usada para a separação por dia.

Timestamps legados sem timezone devem seguir a regra já adotada pelo aplicativo: interpretá-los como UTC. Registros com timestamp inválido não serão descartados; serão exportados com os campos derivados vazios e contabilizados em `warnings` no manifesto.

2. Um ZIP é sempre o artefato final

Independentemente das opções escolhidas, o usuário receberá um único arquivo:

```
smart-events_<evento>_<YYYYMMDD-HHMMSS-bsb>.zip
```

“Consolidado” significa um único CSV de KPIs dentro do ZIP. KPIs, VIPs, alarmes e alertas nunca serão misturados no mesmo CSV, pois possuem esquemas diferentes.

O ZIP deve ser criado com `allowZip64=True` , compressão `ZIP_DEFLATED` , em uma pasta temporária, e publicado no destino somente depois da validação por `os.replace` . Uma falha ou cancelamento não pode deixar um arquivo final parcial.

3. Escopo temporal e consistência

Para evento encerrado, a exportação contém todo o histórico ainda persistido. Para evento ativo, ela representa um snapshot enquanto a coleta continua rodando.

O serviço usará a API de backup online do SQLite para copiar:

- o banco específico do evento, fonte de KPIs, alarmes, alertas, sites e checkpoints;
- o banco global, fonte da configuração, cadastro de VIPs e `vip_measurements` .

As cópias serão feitas para o staging do job e todas as consultas ocorrerão sobre elas. O manifesto registrará `snapshot_started_at` , `event_db_snapshot_at` e `global_db_snapshot_at` . Como os dois bancos

não têm uma transação distribuída, o modal de evento ativo deve usar a expressão “snapshot da gravação” e não prometer atomicidade entre bancos no mesmo milissegundo.

O exportador não deve manter uma transação de leitura longa nos bancos vivos nem pausar o scheduler durante a geração dos CSVs.

4. “Todos os dados” significa todos os dados persistidos do evento

Entram no pacote:

- todas as linhas de `kpi_measurements` do evento;
- todas as linhas de `vip_measurements` filtradas obrigatoriamente por `event_id`;
- todos os alarmes do evento, sem o limite de 500 usado na tela;
- todos os alertas ainda existentes, reconhecidos ou não;
- sites, células, clusters, vínculos de VIPs e checkpoints do evento;
- configuração sanitizada e informações normalizadas da EP.

Não é possível recuperar respostas brutas do OSS que não foram persistidas. O manifesto deve declarar `data_scope: "persisted_event_data"` e o `LEIA-ME.txt` deve explicar essa limitação.

Dados apagados anteriormente por “Limpar dados” também não podem ser recuperados.

5. Segredos nunca entram no pacote

Excluir sempre:

- usuário e senha;
- cookies, tokens e arquivos `session*.json`;
- conteúdo de `credentials.json` e `settings.json`;
- `oss.base_url` e `oss.import_folder`;
- caminhos absolutos da máquina;
- dumps globais de diagnóstico sem associação inequívoca ao evento.

IDs técnicos de task, `obj_no`, EARFCN e identificadores de células podem entrar, pois são necessários para auditoria da coleta, mas devem ser descritos no dicionário de dados.

6. Tecnologia é uma dimensão do KPI, não uma inferência do nome do arquivo

A fonte principal será `kpi_measurements.technology`:

Tecnologia persistida	Família exportada
4G	4G
5G , 5G_NRCELL	5G
5G_NRDUCCELL	5G
vazio ou valor não reconhecido	unknown

A tecnologia da EP enriquece a linha, mas não substitui a tecnologia persistida pelo coletor. Registros sem classificação nunca serão descartados.

7. Valores de KPI preservam o armazenado e expõem o canônico

Cada linha de KPI deve conter:

- `value_stored` : valor exato encontrado no SQLite;
- `value` : valor convertido para a unidade canônica do catálogo;
- `unit` : unidade canônica;
- `oss_unit` : unidade de origem conhecida pelo catálogo;
- `conversion_applied` : `true` OU `false` .

A conversão deve reutilizar `core.kpi_formulas.to_canonical` e o catálogo atual. Métrica desconhecida não pode impedir a exportação: nesse caso `value` repete `value_stored` , as unidades ficam vazias e o manifesto recebe um aviso.

8. Formato dos CSVs

Todos os CSVs devem usar:

- UTF-8 com BOM;
- regras de quoting do módulo `csv` da biblioteca padrão;
- vírgula como separador, declarada no manifesto;
- `\n` como término lógico de linha;
- cabeçalho presente mesmo quando não há registros;
- números com representação invariável;
- ordenação determinística.

Campos textuais iniciados por `=`, `+`, `-`, `@`, tab ou carriage return devem ser neutralizados antes de chegar ao CSV para impedir execução de fórmulas no Excel. O valor original permanece na configuração JSON quando fizer parte dela; o manifesto registra quantas células foram neutralizadas por arquivo.

Contrato do pacote

Estrutura base:

```

smart-events_<evento>_<data>.zip
├─ manifest.json
├─ LEIA-ME.txt
├─ metadados/
│   ├── evento.json
│   ├── ep_normalizada.csv
│   ├── sites.csv
│   ├── celulas.csv
│   ├── clusters.csv
│   └─ dicionario_de_dados.csv
├─ dados/
│   ├── kpis.csv ou kpis/<arquivos conforme as opções>
│   ├── vips.csv
│   ├── alarmes.csv
│   └─ alertas.csv
└─ auditoria/
    ├── checkpoints_de_coleta.csv
    └─ resumo.json

```

Organização escolhida pelo usuário

As duas dimensões são independentes.

Organização temporal

- **consolidated** — um conjunto de KPI para todo o evento; padrão;
- **daily** — um conjunto por **data_brasilia**.

Organização tecnológica

- **combined** — todas as tecnologias juntas; padrão;
- **family** — separa **4G**, **5G** e **unknown**;
- **collector** — separa **4G**, **5G_NRCCELL**, **5G_NRDUCELL** e **unknown**.

Matriz de nomes:

Tempo	Tecnologia	Exemplo
consolidado	juntas	<code>dados/kpis.csv</code>
por dia	juntas	<code>dados/kpis/kpis_2026-08-20.csv</code>
consolidado	família	<code>dados/kpis/kpis_4g.csv</code> , <code>kpis_5g.csv</code>
por dia	família	<code>dados/kpis/2026-08-20/kpis_4g.csv</code>
consolidado	coletor	<code>dados/kpis/kpis_5g_nrcell.csv</code>
por dia	coletor	<code>dados/kpis/2026-08-20/kpis_5g_nrducell.csv</code>

Não criar automaticamente arquivos `part-0001`, `part-0002`. CSV não tem limite de linhas. Antes de iniciar um consolidado muito grande, a interface apenas avisa que o Excel pode não abrir todas as linhas e recomenda “Separar por dia”; a escolha final continua sendo do usuário.

manifest.json

Campos mínimos:

```
{
  "schema_version": 1,
  "package_kind": "smart-events-data-export",
  "event_id": "festa-do-peao-de-barretos-2026",
  "event_name": "Festa do Peão de Barretos 2026",
  "event_status": "ACTIVE",
  "timezone": "America/Sao_Paulo",
  "data_scope": "persisted_event_data",
  "options": {
    "time_partition": "consolidated",
    "technology_partition": "combined",
    "include_vips": true
  },
  "snapshot_started_at": "2026-09-03T15:00:00Z",
  "event_db_snapshot_at": "2026-09-03T15:00:01Z",
  "global_db_snapshot_at": "2026-09-03T15:00:02Z",
  "counts": {},
  "periods": {},
  "files": {},
  "warnings": []
}
```

Cada entrada de `files` conterá caminho relativo, tamanho, número de linhas, SHA-256 e quantidade de células neutralizadas. O manifesto será escrito por último e depois lido pelo próprio validador.

KPIs enriquecidos pela EP

Colunas mínimas de cada CSV de KPI:

```
timestamp_utc
timestamp_brasilia
data_brasilia
event_id
site_id
site_name_ep
cell_id
cell_id_ep
cell_name_ep
metric
value_stored
value
unit
oss_unit
conversion_applied
scope
technology
technology_family
technology_ep
frequency_mhz
earfcn
is_event_site
```

O enriquecimento deve usar mapas imutáveis de site e célula montados uma vez por job. Uma célula sem correspondência na EP continua no CSV, com os campos da EP vazios e aviso agregado no manifesto; não gerar um aviso por linha.

EP normalizada

`metadados/ep_normalizada.csv` terá uma linha por célula:

```
source_row
enodebid
nename
cellid
cellname
latitude
longitude
azimuth
technology
frequency_mhz
dlearfcn
is_event_site
clusters
source_quality
```

Para novos eventos, o importador deve preservar separadamente os identificadores originais da EP em um objeto `ep` no site/célula, sem mudar os campos canônicos já consumidos pelo mapa:

```
{
  "id": "VRX6151",
  "azimuth": 320,
  "tech": "4G",
  "frequency": "1800",
  "earfcn": "12345",
  "ep": {
    "source_row": 2,
    "cellid": "111",
    "cellname": "VRX6151",
    "band": "1800",
    "dlearfcn": "12345"
  }
}
```

O site preservará `ep.enodebid`, `ep.nename`, latitude e longitude originais. Campos numéricos que funcionam como identificadores devem ser serializados como texto para não perder zeros à esquerda.

Eventos antigos não serão reescritos. O exportador reconstruirá o que for possível a partir de `sites` e `cells`, preenchendo `source_quality = "reconstructed"`. Novos eventos terão `source_quality = "preserved"`. Essa diferença aparecerá no manifesto.

O arquivo original da EP não faz parte desta fase: atualmente ele não é retido nem distribuído ao desktop. Preservar o original exigiria um fluxo próprio de upload, armazenamento, sincronização e retenção. A informação necessária da EP será atendida pelo modelo normalizado acima.

Fase única — Exportação completa pelo REC

Explicação simples

O clique no indicador de gravação deixará de significar apenas “apagar”. Ele abrirá uma central de dados do evento, onde o operador poderá exportar tudo ou iniciar a limpeza já existente.

Ao exportar, o aplicativo fará cópias consistentes dos bancos em background, criará os CSVs com as informações da EP, organizará os KPIs conforme as opções de dia e tecnologia, montará um ZIP com manifesto e hashes e disponibilizará o resultado na pasta Downloads. A coleta continuará ativa e a interface mostrará o progresso sem congelar.

Escopo detalhado

Código

1. Preservação das informações da EP

Alterar `server.py` no endpoint `/api/parse-sites` :

- conservar o comportamento atual dos campos canônicos `id`, `name`, `lat`, `lng`, `azimuth`, `tech`, `frequency` e `earfcn`;
- adicionar `site.ep` e `cell.ep` com os valores normalizados dos campos de origem;
- preservar `cellid` mesmo quando `cellname` for escolhido como `cell.id`;
- registrar `source_row` considerando que a linha 1 é o cabeçalho;
- manter identificadores da EP como strings;
- não copiar colunas arbitrárias ou desconhecidas para o evento;
- garantir que os objetos extras sobrevivam à criação, edição, sincronização e empacotamento do evento sem alterar a renderização do mapa.

Atualizar os testes do servidor e do pacote de eventos para provar o round-trip completo:

EP → `/api/parse-sites` → JSON do evento → `.sepack` → importação → exportação de dados

2. Módulo novo `core/event_export.py`

Criar um serviço independente da camada `Api`, usando apenas biblioteca padrão e módulos do projeto.

Responsabilidades:

- `ExportOptions` com validação estrita de `time_partition`, `technology_partition` e `include_vips`;
- `ExportJob` com estado, progresso, fase atual, resultado, aviso e erro sanitizado;
- `EventExportService` com fila e no máximo um worker de geração por processo;
- rejeitar segundo job simultâneo para o mesmo evento;
- permitir cancelamento cooperativo entre lotes e arquivos;
- criar staging dentro de `data/exports/.staging/<job_id>`;
- fazer backup online dos dois SQLite;
- iterar consultas com `fetchmany`, sem `fetchall` sobre conjuntos grandes;
- gerar e fechar cada CSV antes de compactá-lo;
- calcular hashes lendo em chunks;
- validar contagens, hashes e caminhos relativos;
- publicar o ZIP atomicamente em Downloads;
- remover staging em sucesso, falha, cancelamento e inicialização após queda anterior;
- nunca sobrescrever arquivo existente; acrescentar sufixo numérico quando necessário.

Estados públicos:


```
queued → snapshotting → exporting_metadata → exporting_kpis
      → exporting_vips → exporting_alarms → exporting_alerts
      → packing → validating → ready

qualquer estado ativo → cancelling → cancelled
qualquer estado ativo → failed
```

O progresso deve trazer `current`, `total`, `percent`, `dataset` e uma mensagem curta. O cálculo de `total` usa `COUNT(*)` nos snapshots, nunca nos bancos vivos durante a escrita dos CSVs.

3. Consultas de exportação

Não reutilizar getters da tela que impõem janela, agregação ou limite. Criar consultas dedicadas sobre as cópias SQLite:

- KPI: `WHERE event_id = ? ORDER BY timestamp, technology, site_id, cell_id, metric, id ;`
- VIP: `WHERE event_id = ? ORDER BY timestamp, vip_name, id ;`
- alarmes: sem `LIMIT`, ordenados por `arrive_time, csn ;`
- alertas: reconhecidos e não reconhecidos, ordenados por `timestamp, id ;`
- checkpoints: todos do evento, ordenados por OSS, coletor, task e objeto;
- vínculos de VIP: somente do evento;
- sites/células: configuração capturada no snapshot global, com fallback para a tabela do evento.

O atual `get_vip_series` não deve ser usado porque seu `event_id` não restringe a consulta. Criar teste negativo com dois eventos e o mesmo nome de VIP.

Adicionar `vip_id` opcional a `vip_measurements` em uma migração aditiva:

- novas coletas gravam `vip_id` e mantêm `vip_name` como snapshot legível;
- registros antigos tentam resolução por associação inequívoca de evento, nome e task;
- resolução ambígua deixa `vip_id` vazio e gera aviso, sem inventar identidade;
- nenhuma migração destrutiva ou regravação massiva acontece na abertura do aplicativo.

4. Paths, publicação e auditoria

Em `core/paths.py`, adicionar:

- `event_exports_dir()` ;
- `event_export_staging_dir()` ;
- helper compartilhado para resolver Downloads, extraíndo a lógica duplicada de `api/api.py` .

Registrar o último resultado por evento em `data/exports/jobs/<job_id>.json`, com escrita atômica. O registro não deve conter dados de KPI nem informações pessoais; apenas estado, opções, contagens, caminho final, hash do ZIP e timestamps.

Isso permite que o modal informe “última exportação concluída” e que o fluxo de limpeza recomende exportar antes de apagar.

5. API Python

Adicionar em `api/api.py` :

```
preview_event_export(event_id, options) -> dict
start_event_export(event_id, options) -> dict
get_event_export_status(job_id) -> dict
cancel_event_export(job_id) -> dict
open_event_export_folder(job_id) -> dict
get_latest_event_export(event_id) -> dict
```

`preview_event_export` retorna contagens e estimativa conservadora de tamanho, mas não cria snapshots. A resposta deve incluir `excel_row_warning` quando o consolidado estimado ultrapassar 1.048.576 linhas; isso é aviso, não bloqueio.

`start_event_export` valida o evento novamente para evitar TOCTOU entre preview e início. Métodos de status só expõem jobs conhecidos pelo serviço. `open_event_export_folder` só abre a pasta do artefato registrado e pronto; nunca aceita um path vindo do frontend.

6. Ponte JavaScript e mocks

Em `frontend/js/bridge.js` :

- criar wrappers para os seis métodos;
- criar mocks com progresso determinístico;
- oferecer cenários mock de sucesso, falha, cancelamento, pacote grande e evento sem dados;
- manter a tela utilizável quando o backend ainda não estiver pronto em modo navegador.

Criar `frontend/js/event_export.js` para isolar:

- abertura/fechamento do modal;
- estado das opções;
- debounce do preview;
- polling do job;
- progresso e cancelamento;
- tratamento do resultado;
- encerramento de timers ao trocar de evento ou sair da tela.

`frontend/js/app.js` apenas inicializa e notifica o módulo ao mudar o evento.

7. Contrato e documentação

Adicionar documentação do formato em `docs/exportacao-de-dados.md` , incluindo:

- árvore do ZIP;
- schemas dos CSVs;
- timezone de Brasília;
- definição de consolidado/dia e família/coletor;
- unidades armazenada e canônica;
- dados incluídos e excluídos;
- como conferir SHA-256;
- limitações de dados históricos e respostas brutas.

Atualizar `README.md` com uma descrição curta e link para a documentação canônica.

Interface

1. Comportamento do indicador

Manter o componente atual no cabeçalho e torná-lo acessível por teclado:

- evento ativo: `REC <tamanho>` com ponto vermelho;
- evento histórico: `DADOS <tamanho>` sem animação de gravação;
- nenhum evento: oculto;
- `role="button"`, `tabindex="0"`, nome acessível e ativação por Enter/Espaço.

O clique não abre mais diretamente o modal destrutivo. Abre o novo modal "Dados do evento".

2. Modal "Dados do evento"

Primeiro nível:

```
Dados do evento
Festa do Peão de Barretos 2026
486 MB gravados

[ Exportar dados ]
[ Limpar dados ]

Última exportação: 03/09/2026 12:04 — concluída
```

- "Exportar dados" é ação primária;
- "Limpar dados" usa estilo destrutivo e abre a dupla confirmação existente;
- se nunca houve exportação, o primeiro modal de limpeza mostra "Nenhuma exportação concluída" e oferece "Exportar antes de limpar";
- não adicionar confirmação extra além das duas etapas destrutivas atuais;
- fechar por botão, clique no backdrop e Escape;
- prender foco dentro do modal enquanto aberto e devolver foco ao `REC` ao fechar.

3. Configuração da exportação

Segundo nível ou painel expandido:

Conteúdo
☒ KPIs
☒ VIPs
☒ Alarmes
☒ Alertas
☒ Informações da EP

Organização dos KPIs
(•) Consolidado
() Separar por dia

Tecnologia
(•) Manter juntas
() Separar em 4G e 5G
() Separar por tecnologia de coleta

KPIs, alarmes, alertas, evento e EP fazem parte do pacote completo e não precisam ser desmarcáveis. O controle de conteúdo efetivamente opcional será **VIPs**, acompanhado de aviso de dados pessoais. Os checkboxes fixos podem ser apresentados como uma lista informativa para não sugerir opções que não existem.

Padrões:

- **Consolidado** ;
- **Manter juntas** ;
- incluir VIPs;
- sempre incluir EP normalizada.

Exibir preview com:

- número de KPIs, VIPs, alarmes e alertas;
- menor e maior timestamp;
- número de dias no horário de Brasília;
- tecnologias encontradas;
- estimativa de tamanho;
- aviso de snapshot quando o evento estiver ativo;
- aviso de Excel quando aplicável;
- qualidade da EP: preservada ou reconstruída.

4. Progresso

Durante o job:

- desabilitar mudanças nas opções;
- manter o dashboard e a coleta utilizáveis;
- mostrar fase, percentual, conjunto atual e contagem;
- permitir “Cancelar exportação” até o início de `validating` ;
- não permitir iniciar outra exportação do mesmo evento;
- se o modal for fechado, manter um indicador discreto no `REC` e restaurar o progresso ao reabrir.

Ao concluir:

Exportação concluída
620 MB · 8.464.853 registros

[Abrir pasta] [Fechar]

Em falha, mostrar mensagem acionável e manter um botão para tentar novamente. Não exibir traceback, paths temporários ou segredos.

5. Ajustes visuais

Alterar `frontend/index.html` e `frontend/css/main.css` usando o design system atual:

- largura suficiente para as opções sem ultrapassar a viewport;
- scroll apenas no corpo do modal em telas baixas;
- estados de foco visíveis;
- barra de progresso com texto, não apenas cor;
- alerta amarelo para evento ativo/Excel;
- alerta vermelho apenas para limpeza e falha;
- layout responsivo em 1366×768, 1920×1080 e escala de tela Windows de 125%.

Testes

1. `tests/test_event_export.py` — contrato e motor

Cobrir no mínimo:

Teste	Garantia
test_export_contains_all_expected_datasets	árvore mínima do ZIP
test_export_filters_every_dataset_by_event_id	nenhum vazamento entre eventos
test_export_includes_acknowledged_alerts	exporta histórico, não só ativos
test_export_does_not_apply_alarm_ui_limit	mais de 500 alarmes são preservados
test_export_writes_headers_for_empty_dataset	CSV vazio continua utilizável
test_export_uses_brasilia_for_daily_partition	dia calculado no timezone correto
test_export_handles_brasilia_midnight_boundary	UTC próximo da meia-noite cai no dia local certo
test_legacy_naive_timestamp_is_treated_as_utc	compatibilidade histórica
test_invalid_timestamp_is_exported_with_warning	dado ruim não é perdido
test_combined_kpi_export_creates_one_file	consolidado/juntas
test_daily_kpi_export_creates_one_file_per_brasilia_day	separação temporal
test_family_partition_combines_nrcell_and_nrducell	família 5G correta
test_collector_partition_separates_5g_types	modo avançado correto
test_unknown_technology_is_never_dropped	arquivo/linha unknown
test_kpi_exports_stored_and_canonical_values	fidelidade e unidade
test_kpi_is_enriched_from_ep	join EP por célula/site
test_unmapped_cell_remains_in_export	enriquecimento não remove dado
test_old_event_reconstructs_ep_with_quality_flag	compatibilidade histórica
test_manifest_counts_match_csv_rows	contagens auditáveis
test_manifest_hashes_match_every_file	integridade
test_csv_neutralizes_spreadsheet_formulas	segurança ao abrir no Excel
test_export_never_contains_credentials_or_machine_paths	ausência de segredos
test_failed_export_does_not_publish_partial_zip	atomicidade
test_cancelled_export_removes_staging	cancelamento limpo
test_existing_destination_gets_unique_suffix	não sobrescreve arquivo
test_active_writes_continue_during_export	backup online sem bloquear coleta
test_large_export_uses_bounded_memory	leitura em lotes

O teste de memória pode ser marcado como `performance` e rodar fora da suíte rápida, usando dados sintéticos suficientes para detectar `fetchall` ou construção integral do CSV em memória.

2. Migração e banco

Atualizar `tests/test_database.py` :

- banco novo possui `vip_measurements.vip_id` ;
- banco legado recebe a coluna sem perder linhas;
- gravação nova persiste `vip_id` e `vip_name` ;
- resolução histórica só preenche identidade inequívoca;
- nenhuma consulta da exportação cria ou altera dados no snapshot fonte.

3. EP e servidor

Atualizar `tests/test_server_parse_sites.py` :

- `cellid` e `cellname` são preservados separadamente;
- `source_row` está correto;
- banda e DLEARFCN mantêm representação textual;
- aliases continuam funcionando;
- EP sem tecnologia mantém a inferência atual;
- coluna desconhecida não é copiada;
- evento legado sem objeto `ep` continua válido.

Atualizar `tests/test_event_package.py` e testes de importação para provar que os novos objetos `ep` sobrevivem ao `.sepack` e à sincronização.

4. API

Atualizar `tests/test_api.py` ou criar `tests/test_event_export_api.py` :

- preview de evento inexistente falha sem criar job;
- opções inválidas são rejeitadas;
- start devolve `job_id` e não bloqueia a chamada;
- estados seguem transições válidas;
- segundo job do mesmo evento é recusado;
- cancelamento é idempotente;
- pasta só pode ser aberta para job `ready` ;
- erro interno volta sanitizado;
- último export concluído é encontrado por evento.

5. Frontend

Criar `tests/test_frontend_event_export_ui.py` :

- o `REC` abre "Dados do evento", não o modal de limpeza;
- "Limpar dados" ainda alcança as duas confirmações existentes;
- defaults são consolidado, tecnologias juntas e VIPs incluídos;
- opções são enviadas com os valores contratuais;
- troca de opção refaz o preview sem duplicar chamadas;
- polling termina em `ready`, `failed` e `cancelled`;
- timers são removidos ao trocar de evento;
- evento histórico mostra `DADOS` e permite exportação;
- Escape, foco e teclado funcionam;
- textos obrigatórios de snapshot, dados pessoais e limite do Excel existem;
- HTML não injeta nome do evento ou mensagens do backend sem escape.

Atualizar os mocks do bridge para permitir um smoke visual no navegador sem Python.

6. Build e regressão

- atualizar `core/self_test.py` para verificar que `frontend/js/event_export.js` está no bundle;
- confirmar que o import estático inclui `core/event_export.py` no PyInstaller;
- rodar os testes existentes de coleta, banco, frontend, pacote e instalador;
- confirmar que o download de logs e de alertas continua usando a pasta correta após extrair o helper de Downloads;
- confirmar que limpar histórico ainda remove KPI, alarmes e alertas conforme o contrato atual.

Como validar e testar

1. Testes automatizados direcionados

Executar primeiro:

```
.venv\Scripts\python.exe -m pytest `
tests/test_event_export.py `
tests/test_event_export_api.py `
tests/test_frontend_event_export_ui.py `
tests/test_server_parse_sites.py `
tests/test_event_package.py `
tests/test_database.py -q
```

Se a API permanecer em `tests/test_api.py`, substituir o arquivo inexistente na lista acima pelo arquivo efetivamente usado.

Depois executar a regressão sem VPN:


```
.venv\Scripts\python.exe -m pytest -m "not vpn and not performance"
```

E separadamente:

```
.venv\Scripts\python.exe -m pytest -m performance tests/test_event_export.py
```

Esperado:

- zero falhas;
- zero warnings novos de SQLite bloqueado;
- teste grande não cresce memória proporcionalmente ao total de linhas;
- nenhuma chamada de rede é feita pela suíte de exportação.

2. Validação do contrato do ZIP

Usar um diretório de dados descartável e um evento pequeno conhecido:

1. inserir uma linha representativa em cada conjunto;
2. incluir timestamps dos dois lados da meia-noite de Brasília;
3. incluir KPIs 4G , 5G_NRCCELL , 5G_NRDUCELL e tecnologia vazia;
4. incluir uma célula mapeada e outra ausente na EP;
5. gerar as seis combinações relevantes de tempo/tecnologia;
6. abrir cada ZIP e conferir que nenhum caminho é absoluto ou contém .. ;
7. recalcular SHA-256 de todos os arquivos;
8. contar as linhas dos CSVs e comparar com o manifesto;
9. confirmar que a união dos arquivos particionados tem exatamente as mesmas linhas do consolidado;
10. pesquisar no pacote por username, senha, token, URL interna e raiz do workspace.

Critério: a única diferença entre consolidado e particionado é a organização dos arquivos; dados, valores e contagens globais são idênticos.

3. Validação do horário de Brasília

Criar registros de prova:

UTC	Brasília esperado	Arquivo diário esperado
2026-08-21T02:59:59Z	2026-08-20T23:59:59-03:00	2026-08-20
2026-08-21T03:00:00Z	2026-08-21T00:00:00-03:00	2026-08-21

Validar o mesmo resultado com o Windows configurado temporariamente em outro timezone ou, nos testes, monkeypatchando o timezone local. O resultado não pode mudar.

4. Validação visual no navegador

Abrir `frontend/index.html` em modo mock e verificar:

1. `REC` recebe foco e abre o modal por clique, Enter e Espaço;
2. o foco permanece dentro do modal e retorna ao indicador ao fechar;
3. Escape e backdrop fecham apenas o nível atual;
4. "Limpar dados" preserva a dupla confirmação;
5. as opções padrão estão corretas;
6. alternar dia/tecnologia atualiza nomes esperados e preview;
7. evento ativo mostra aviso de snapshot;
8. evento histórico usa `DADOS`, sem ponto pulsante;
9. pacote grande mostra recomendação de separação por dia sem bloquear o consolidado;
10. progresso, cancelamento, falha e sucesso são legíveis;
11. testar 1366×768, 1920×1080 e simulação de 125% de escala;
12. confirmar ausência de corte, sobreposição, scroll horizontal e mudança brusca do header.

Capturar ao menos screenshots dos estados:

- modal inicial;
- opções e preview;
- exportação em andamento;
- conclusão;
- aviso de falha;
- confirmação de limpeza sem exportação anterior.

5. Validação funcional no aplicativo

Executar com um banco descartável e depois com uma cópia de um banco grande real:

1. iniciar um evento mock e aguardar novas coletas;
2. clicar no `REC` , iniciar exportação consolidada e continuar navegando no mapa e nos gráficos;
3. acompanhar o tamanho do WAL e verificar ausência de congelamento prolongado;
4. fechar e reabrir o modal durante o job: o progresso deve continuar;
5. cancelar no meio dos KPIs e confirmar que não existe ZIP final;
6. repetir e aguardar sucesso;
7. abrir a pasta pelo botão da interface;
8. abrir os CSVs menores no Excel e conferir acentos, colunas, datas e valores;
9. importar o consolidado em uma ferramenta que suporte mais de um milhão de linhas;
10. comparar amostras de `value_stored` diretamente com o snapshot SQLite;
11. comparar `value` com os mesmos KPIs exibidos pelo dashboard;
12. encerrar o evento e repetir pelo estado `DADOS` ;
13. iniciar limpeza sem exportação anterior e conferir a recomendação;
14. concluir uma exportação, voltar à limpeza e conferir a data da última exportação.

Durante toda a prova, os ciclos do scheduler devem continuar sem `database is locked` e sem perda de lotes.

6. Validação da EP

1. criar evento novo importando `ep_default.xlsx` ;
2. salvar, sincronizar e, quando aplicável, transportar pelo `.sepack` ;
3. confirmar no JSON final a presença separada de `cellid` , `cellname` , banda e DLEARFCN;
4. exportar o evento;
5. comparar `ep_normalizada.csv` com a EP de entrada campo a campo;
6. conferir o enriquecimento das linhas de KPI correspondentes;
7. repetir com evento antigo sem objeto `ep` e verificar `source_quality = reconstructed` ;
8. confirmar que campos desconhecidos da planilha não aparecem por acidente no pacote.

7. Validação no executável empacotado

1. gerar build de desenvolvimento;
2. rodar o self-test do bundle;
3. instalar/executar em diretório sem o repositório disponível;
4. exportar para a pasta Downloads real do usuário;
5. confirmar que nenhuma dependência de desenvolvimento é necessária;
6. cancelar e repetir uma exportação grande;
7. reiniciar o app após interrupção forçada e verificar limpeza do staging órfão;
8. validar novamente download de logs, alertas e abertura da pasta.

Critérios de aceite da fase

A fase está concluída quando:

- o clique no `REC` abre a central de dados e não a exclusão diretamente;
- eventos ativos e históricos podem ser exportados;
- o pacote contém todos os conjuntos persistidos do evento e a EP normalizada;
- consolidado e separado por dia produzem a mesma população de KPIs;
- os três modos tecnológicos funcionam sem perder `unknown` ;
- todo agrupamento diário usa exclusivamente o horário de Brasília;
- valores armazenados e canônicos estão documentados e corretos;
- nenhum dado de outro evento aparece, especialmente nas medições de VIP;
- nenhuma credencial, token, URL interna ou path da máquina aparece no ZIP;
- a coleta e a interface continuam responsivas durante uma exportação grande;
- falha e cancelamento não publicam arquivos parciais;
- manifesto, hashes, contagens e períodos são verificáveis;
- dados da EP novos são preservados e eventos legados recebem fallback explícito;
- a dupla confirmação de limpeza continua funcionando;
- testes direcionados, regressão sem VPN, smoke visual e prova no executável passam.

Sugestão de commit

```
feat: export event data from recording panel
```